

Claude Code

Guía rápida en la Terminal (CLI)

La guía más fácil de entender del mundo — edición ampliada

CMD · PowerShell · Bash

Sin relleno

Copiar & pegar

Contenido

1. ¿Qué es Claude Code en 1 frase?
2. Instalación y Login
3. Los 5 comandos esenciales (el 90% del tiempo)
4. Modo Interactivo vs Modo Comando Único
5. ★ CLI vs App de Escritorio: lo que solo puedes hacer en terminal
6. Referencia completa: comandos y flags del CLI
7. Comandos slash (dentro de la sesión)
8. Atajos de teclado y trucos de escritura
9. Ejemplo práctico: cazar un bug paso a paso
10. Recetas listas para copiar (one-liners)
11. Personalización pro: CLAUDE.md, comandos propios, agentes, hooks y MCP
12. Tips de oro: menos tokens, mejores respuestas
13. Chuleta final

1. ¿Qué es Claude Code en 1 frase?

Es un programador con IA que vive en tu terminal: lee tu proyecto, edita archivos, ejecuta comandos, usa git y resuelve tareas completas con solo pedirselo en lenguaje natural.

No es un chat donde copias y pegas código. Claude Code **trabaja directamente sobre tus archivos**, como un compañero sentado frente a tu PC: explora el código él solo, propone cambios, los aplica (con tu permiso), corre los tests y hasta hace el commit.

2. Instalación y Login (desde cero)

Paso 1 — Instalar

Windows (PowerShell):

```
irm https://claude.ai/install.ps1 | iex
```

macOS / Linux:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Alternativa con npm (si ya tienes Node.js 18+):

```
npm install -g @anthropic-ai/claude-code
```

⚠ Tip Windows

Si algo falla, instala **Git for Windows** (git-scm.com) primero: Claude Code lo usa internamente. Funciona en CMD, PowerShell y Git Bash.

Paso 2 — Entrar a tu proyecto y arrancar

```
cd mi-proyecto
claude
```

La **primera vez** se abre el navegador para iniciar sesión:

Opción	Para quién
Cuenta Claude (Pro / Max)	La opción normal: usas tu suscripción mensual.
Anthropic Console (API key)	Si pagas por uso (facturación por tokens).

Paso 3 — Verificar que todo está bien

```
claude --version    # ¿Está instalado?
claude doctor       # Diagnóstico completo de la instalación
claude update       # Actualizar a la última versión
```

✓ **Listo**

Eso es todo. Dentro de la sesión, `/login` y `/logout` sirven para cambiar de cuenta.

3. Los 5 comandos esenciales (el 90% de tu día)

#	Comando	Qué hace	En peras y manzanas 🍌🍏
1	<code>claude</code>	Abre una sesión interactiva	"Hola, vengo a trabajar"
2	<code>claude -c</code>	Continúa tu última conversación	"¿En qué estábamos ayer?"
3	<code>/clear</code>	Borra la memoria de la sesión	"Borrón y cuenta nueva" (tema nuevo = <code>/clear</code>)
4	<code>/compact</code>	Resume la conversación y libera memoria	"Hazme un resumen y sigamos"
5	<code>claude -p "..."</code>	Pregunta única: responde y se cierra	"Una consulta rápida y me voy"

Ejemplos reales de cada uno:

```
claude                                # Sesión normal de trabajo
claude -c                            # Retomar donde quedaste
claude -p "¿Qué hace este script?" < app.py  # Respuesta rápida sin abrir sesión
```

🎁 Bonus — 3 teclas que te salvarán la vida

`Esc` detiene a Claude a mitad de tarea ("¡para, para!") · `Shift` + `Tab` cambia de modo: normal → auto-aceptar → **modo plan** · `@` menciona un archivo exacto: `@src/app.py`

4. Modo Interactivo vs Modo Comando Único

🗨️ Interactivo — `claude`

Una **conversación continua**: Claude recuerda todo lo hablado en la sesión. Tu modo por defecto.

```
claude
> revisa el login, creo que hay un bug
> ok, arréglalo
> ahora agrégale un test
```

Úsalo para: desarrollar, depurar, refactorizar — cualquier tarea con idas y vueltas.

⚡ Comando único — `claude -p`

Una **pregunta** → una **respuesta** → **se cierra**. Sin conversación. Perfecto para scripts y pipes.

```
claude -p "Resume los TODOs del proyecto"
git diff | claude -p "Explica estos cambios"
type error.log | claude -p "¿Causa raíz?"
```

Úsalo para: consultas rápidas, automatización, integrarlo en otros scripts.

💡 Regla mental

¿Vas a responderle algo después? → **Interactivo**. ¿Solo quieres una respuesta? → `-p`.

5. ★ CLI vs App de Escritorio: lo que solo puedes hacer en terminal

La app de escritorio y la CLI comparten el mismo motor (modelos, permisos, CLAUDE.md, MCP...). La app gana en comodidad visual; pero la terminal tiene **superpoderes de automatización e integración** que la app no puede igualar:

① Modo headless: Claude dentro de tus scripts

Con `-p` Claude se convierte en **un comando más de tu sistema**, combinable con cualquier otro programa:

```
# La salida de un comando es la entrada de Claude (pipes):
git diff | claude -p "Escribe un mensaje de commit para estos cambios"
type error.log | claude -p "Encuentra la causa raíz de este error"      # CMD
Get-Content error.log | claude -p "Encuentra la causa raíz"             # PowerShell

# Y la salida de Claude alimenta a otros programas:
claude -p "Lista las funciones sin docstring en polyx/" > pendientes.txt
```

② Salida JSON estructurada (para programas, no humanos)

```
claude -p "Resume este proyecto" --output-format json
# Devuelve JSON con el resultado, costo, duración y nº de turnos → fácil de parsear
claude -p "..." --output-format stream-json  # streaming línea a línea
```

③ Funciona donde no hay escritorio

Por SSH en un **servidor remoto**, dentro de **WSL**, en un **contenedor Docker**, en una Raspberry Pi... Si hay terminal, hay Claude Code. La app de escritorio necesita Windows/macOS con interfaz gráfica.

```
ssh usuario@servidor
cd /var/www/mi-app && claude      # Claude trabajando en el servidor, no en tu PC
```

④ Automatización programada y CI/CD

```
# Programador de tareas de Windows / cron en Linux:
claude -p "Revisa los logs de hoy y resume errores nuevos" >> reporte_diario.txt

# GitHub Actions: revisión automática de cada Pull Request
# (acción oficial: anthropics/claude-code-action)
```

⑤ Control fino por flags en cada invocación

```
# Solo lectura: revisar sin tocar nada (ideal para auditorías)
claude -p "Audita la seguridad de auth.py" --allowedTools "Read,Grep,Glob"

# Limitar vueltas en automatización (que no se eternice):
claude -p "Arregla los tests" --max-turns 10

# Elegir modelo por tarea: barato para lo simple, potente para lo difícil
claude --model haiku -p "Corrige los typos del README"
```

⑥ Varias sesiones en paralelo, a tu manera

Abre 2–3 terminales (o paneles de tmux) con un Claude en cada uno: uno arregla un bug, otro escribe documentación, otro revisa. Con **git worktrees** cada uno trabaja en su propia copia del repo sin pisarse:

```
git worktree add ../proyecto-fix rama-fix
cd ../proyecto-fix && claude          # Claude nº 2, aislado del primero
```

Capacidad	Terminal (CLI)	App de escritorio
Conversar, editar código, git, MCP, CLAUDE.md	✓	✓
Modo headless <code>-p</code> (scripts)	✓	✗
Pipes con otros comandos (<code>git diff claude</code>)	✓	✗
Salida JSON parseable	✓	✗
SSH / servidores / Docker / WSL	✓	✗
CI/CD y tareas programadas (cron)	✓	✗
Flags por invocación (modelo, herramientas, turnos)	✓	✗
Interfaz gráfica cómoda, diffs visuales	— (es texto)	✓

 **En resumen**

La app de escritorio es **Claude como asistente**; la CLI es además **Claude como herramienta de tu sistema**: programable, combinable y presente en cualquier máquina.

6. Referencia completa: comandos y flags del CLI

Comandos

Comando	Qué hace
<code>claude</code>	Abre sesión interactiva en la carpeta actual.
<code>claude "consulta"</code>	Abre sesión interactiva empezando con esa petición.
<code>claude -p "consulta"</code>	Modo headless: responde e termina (ideal scripts).
<code>claude -c</code> / <code>--continue</code>	Continúa la conversación más reciente.
<code>claude -r</code> / <code>--resume</code>	Muestra conversaciones pasadas para elegir cuál retomar.
<code>claude update</code>	Actualiza Claude Code a la última versión.
<code>claude doctor</code>	Diagnostica la instalación (¡tu primer paso si algo falla!).
<code>claude mcp add / list / remove</code>	Gestiona servidores MCP (conectores externos).
<code>claude --version</code>	Muestra la versión instalada.

Flags más útiles

Flag	Qué hace	Ejemplo
<code>--model</code>	Elige el modelo de esa sesión	<code>claude --model opus</code>
<code>--add-dir</code>	Da acceso a carpetas extra fuera del proyecto	<code>claude --add-dir ../otra-carpeta</code>
<code>--output-format</code>	Formato de salida en modo <code>-p</code> : <code>text</code> , <code>json</code> , <code>stream-json</code>	<code>claude -p "..." --output-format json</code>
<code>--max-turns</code>	Límite de turnos en modo <code>-p</code> (no se eterniza)	<code>--max-turns 5</code>
<code>--allowedTools</code>	Herramientas permitidas sin preguntar	<code>--allowedTools "Read,Grep"</code>
<code>--disallowedTools</code>	Herramientas prohibidas	<code>--disallowedTools "Bash"</code>
<code>--permission-mode</code>	Modo de permisos inicial (p. ej. <code>plan</code>)	<code>claude --permission-mode plan</code>
<code>--append-system-prompt</code>	Añade instrucciones de sistema (modo <code>-p</code>)	<code>--append-system-prompt "Responde en español"</code>
<code>--verbose</code>	Muestra el detalle turno a turno (depurar)	<code>claude --verbose</code>

⚠ Existe, pero con cuidado

`--dangerously-skip-permissions` hace que Claude no pida permiso para nada. Útil en contenedores aislados para automatización; **nunca** lo uses en tu máquina con datos importantes.

7. Comandos slash (dentro de la sesión)

Se escriben empezando con `/` en la sesión interactiva. Los imprescindibles:

Comando	Qué hace
<code>/help</code>	Lista todos los comandos disponibles (la fuente de la verdad).
<code>/init</code>	Analiza tu proyecto y crea el archivo CLAUDE.md (memoria permanente).
<code>/clear</code>	Borra todo el historial: empezar de cero (tema nuevo).
<code>/compact</code>	Resume la conversación para liberar contexto. Acepta foco: <code>/compact conserva lo del bug del login</code> .
<code>/model</code>	Cambia de modelo en caliente (Opus ↔ Sonnet ↔ Haiku).
<code>/cost</code> · <code>/usage</code>	Cuánto llevas gastado / cuánto queda de tu límite del plan.
<code>/context</code>	Mapa visual de qué está ocupando la memoria de contexto.
<code>/memory</code>	Edita los archivos CLAUDE.md (memoria del proyecto o global).
<code>/permissions</code>	Gestiona qué herramientas/comandos puede usar sin preguntar.
<code>/review</code>	Revisión de código del diff o de una Pull Request.
<code>/rewind</code>	⏮ Vuelve a un punto anterior (checkpoint): deshace cambios de código y/o conversación.
<code>/resume</code>	Cambia a otra conversación pasada sin salir.
<code>/agents</code>	Crea/gestiona subagentes especializados (revisor, documentador...).
<code>/mcp</code>	Estado y autenticación de los servidores MCP conectados.
<code>/hooks</code>	Configura acciones automáticas (p. ej. formatear tras cada edición).
<code>/config</code>	Ajustes generales (tema, notificaciones...).
<code>/export</code>	Exporta la conversación (portapapeles o archivo).
<code>/doctor</code> · <code>/status</code>	Salud de la instalación · cuenta, modelo y estado actual.
<code>/vim</code> · <code>/terminal-setup</code>	Modo de edición vim · configura <code>Shift</code> + <code>Enter</code> para salto de línea.
<code>/bug</code>	Reporta un problema de Claude Code a Anthropic.
<code>/login</code> · <code>/logout</code>	Cambiar de cuenta / cerrar sesión.

8. Atajos de teclado y trucos de escritura

Tecla / símbolo	Qué hace
<code>Esc</code>	Interrumpe a Claude a mitad de tarea (lo redibujas y sigue).
<code>Esc</code> <code>Esc</code>	Doble Esc: salta atrás en la conversación para editar un mensaje anterior o restaurar un checkpoint.
<code>Shift</code> + <code>Tab</code>	Cicla modos de permiso: normal → auto-aceptar ediciones → modo plan .
<code>@</code>	Menciona archivos o carpetas con autocompletado: <code>@polyx/detector/runner.py</code> .
<code>!</code>	Modo bash: ejecuta un comando de shell directo (<code>!git status</code>) y su salida queda en el contexto.
<code>#</code>	Empieza el mensaje con # para guardarlo como memoria en CLAUDE.md: <code># siempre usa pathlib, no strings</code> .
<code>↑</code> / <code>↓</code>	Navega el historial de tus mensajes anteriores.
<code>Tab</code>	Autocompleta rutas de archivos.
<code>Ctrl</code> + <code>R</code>	Muestra la transcripción completa/detallada de lo que hizo Claude.
<code>Ctrl</code> + <code>C</code>	Cancela la entrada actual (dos veces: salir).
<code>Ctrl</code> + <code>D</code>	Salir de Claude Code.
<code>Ctrl</code> + <code>V</code>	Pega una imagen del portapapeles (¡screenshots de errores!).
<code>\</code> + <code>Enter</code>	Salto de línea sin enviar (o <code>Shift</code> + <code>Enter</code> tras <code>/terminal-setup</code>).

Truco poco conocido

Claude Code **ve imágenes**: pega un pantallazo del error o de un diseño con `Ctrl` + `V` y pídele "haz que se vea así" o "explica este error".

9. Ejemplo práctico: cazar un bug paso a paso 🐛

Imagina que `utils/calculo.py` devuelve resultados incorrectos.

1 Entra al proyecto y abre Claude

```
cd mi-proyecto
claude
```

2 Describe el bug señalando el archivo (síntoma + dónde + qué esperabas)

```
> En @utils/calculo.py la función calcular_promedio() devuelve 0
    cuando la lista tiene un solo elemento. Debería devolver ese elemento.
    Primero diagnostica la causa, NO lo arregles todavía.
```

3 Claude lee el archivo y explica la causa. Si te convence:

```
> Perfecto, arréglalo y agrega un test que cubra ese caso.
```

4 Verifica que el arreglo funciona de verdad

```
> Corre los tests y muéstrame el resultado.
```

5 Si todo pasa en verde, cierra el ciclo

```
> Haz commit con un mensaje descriptivo.
```

🏆 ¿Por qué este orden funciona tan bien?

`@archivo` → va directo al grano, sin buscar a ciegas. · **"Diagnostica primero"** → evita parches que tapan el síntoma sin curar la causa. · **"Corre los tests"** → nada de "debería funcionar": pruebas reales.

📄 Y si algo sale mal...

`/rewind` (o doble `Esc`) restaura el código al punto anterior al cambio. Experimenta sin miedo.

10. Recetas listas para copiar (one-liners)

```
# --- Git ---
git diff | claude -p "Escribe el mensaje de commit (convencional, en español)"
git log --oneline -20 | claude -p "Resume qué se hizo esta semana"

# --- Entender código ajeno ---
claude -p "Explica la arquitectura de este proyecto en 10 líneas"
claude -p "¿Dónde se calcula el diámetro en  $\mu\text{m}$ ? Dame archivo y línea"

# --- Logs y errores ---
type error.log | claude -p "Agrupar los errores por causa y ordena por frecuencia"

# --- Documentación ---
claude -p "Genera docstrings estilo Google para @polyx/core/metrics.py"

# --- Revisión de solo lectura (no toca nada) ---
claude -p "Busca bugs de manejo de rutas en polyx/" --allowedTools "Read,Grep,Glob"

# --- Continuar la última sesión, pero en modo script ---
claude -c -p "¿Quedó algo pendiente de la última sesión?"

# --- Datos estructurados para otro programa ---
claude -p "Lista los TODO/FIXME como JSON" --output-format json > deuda.json
```


12. Tips de oro 🏆 (menos tokens, mejores respuestas)

💰 Para no gastar de más

Tip	Por qué
<code>/clear</code> entre tareas distintas	El historial viejo se reenvía con cada mensaje. Tema nuevo = sesión limpia = más barato.
<code>/compact</code> en pausas naturales	Resume lo hablado y libera memoria sin perder el hilo.
Menciona archivos con <code>@</code>	Evita que Claude explore medio proyecto buscando dónde está el código.
No pegues archivos gigantes	Claude puede leerlos él mismo — y solo la parte que necesita.
Modelo según la tarea (<code>/model</code>)	Haiku para lo simple y barato; Opus para lo difícil.
<code>/cost</code> , <code>/usage</code> y <code>/context</code>	Mide tu gasto y qué está llenando el contexto.

🎯 Para obtener mejores respuestas

Tip	Ejemplo / detalle
Sé específico, no genérico	❌ "arregla el código" → ✅ "el login falla con emails en mayúsculas, en @auth/login.py"
Crea CLAUDE.md con <code>/init</code>	Se lo explicas una vez, no cada día. Añade reglas al vuelo con <code>#</code> .
Modo Plan para tareas grandes	<code>Shift</code> + <code>Tab</code> ×2: Claude propone el plan ANTES de tocar código. Tú apruebas. Cero sorpresas.
Diagnóstico antes de arreglo	"Primero dime la causa" produce arreglos de raíz, no parches.
Exige verificación	Termina con "corre los tests y muéstrame el output".
<code>Esc</code> sin miedo	Si va por mal camino, interrúmpelo y redirige: más barato que dejarlo terminar mal.
Una tarea a la vez	"Arregla X" y luego "documenta Y" — mejor que todo en un mensaje kilométrico.
Pega screenshots	<code>Ctrl</code> + <code>V</code> con la imagen del error o del diseño esperado vale más que mil palabras.

🏆 La regla de oro final

Trátalo como a un colega nuevo muy capaz: dale contexto (CLAUDE.md), señala el archivo exacto (`@`), pide una cosa a la vez y verifica el resultado. Hazlo así y gastarás menos y obtendrás más.

13. Chuleta final para imprimir

Desde la terminal

```
claude          # Sesión interactiva
claude -c        # Continuar la última
claude -r        # Elegir una pasada
claude -p "... "  # Pregunta única
claude --model opus # Elegir modelo
claude mcp list   # Conectores MCP
claude update     # Actualizar
claude doctor     # Diagnóstico
```

Teclas

```
Esc           # Frenar a Claude
Esc Esc       # Volver atrás / editar
Shift+Tab     # normal/auto/plan
@archivo      # Señalar archivo
!comando      # Shell directo
#texto        # Guardar en memoria
Ctrl+V        # Pegar imagen
Ctrl+R        # Ver transcripción
```

Dentro de la sesión

```
/init          # Crear CLAUDE.md
/clear          # Empezar de cero
/compact        # Resumir y liberar
/model          # Cambiar modelo
/cost           # Gasto de la sesión
/usage          # Límites del plan
/context        # Qué llena la memoria
/memory         # Editar CLAUDE.md
/permissions    # Permisos de tools
/review         # Revisar código/PR
/rewind         #  Checkpoint
/agents         # Subagentes
/mcp            # Estado MCP
/hooks          # Automatizaciones
/export         # Exportar charla
/help           # Todos los comandos
```